Module 5 – Neural Network

Solved Jupyter Notebook • Completed code, explanations, training results and error graph

1. Overview

This notebook builds a feed-forward neural network using NumPy. It has 3 input features, a hidden layer with 4 neurons, and 1 output neuron. The network learns the supplied sample dataset over 5,000 training epochs using sigmoid activation and backpropagation.

2. Complete notebook code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Sigmoid activation function and derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Sample input dataset
X = np.array([[0, 0, 1],
              [0, 1, 1],
              [1, 0, 1],
              [1, 1, 1]])

# Sample output dataset
y = np.array([[0],
              [1],
              [1],
              [0]])

# Initialize weights
np.random.seed(1)
w0 = 2 * np.random.random((3, 4)) - 1
w1 = 2 * np.random.random((4, 1)) - 1

# Train the neural network
errors = []
for i in range(5000):
    # Forward propagation
    a0 = X
    a1 = sigmoid(np.dot(a0, w0))
    a2 = sigmoid(np.dot(a1, w1))

    # Output error: prediction minus actual
    output_error = a2 - y

    errors.append({
        "epochs": i,
        "error": np.mean(np.abs(output_error))
    })

    if (i % 500) == 0:
        print("Error after " + str(i) + " epochs:" +
              str(np.mean(np.abs(output_error))))

    # Backpropagation
    layer2_delta = output_error * sigmoid_derivative(a2)
    layer1_error = layer2_delta.dot(w1.T)
    layer1_delta = layer1_error * sigmoid_derivative(a1)

    # Update weights using gradient descent
    w1 -= np.dot(a1.T, layer2_delta)
    w0 -= np.dot(a0.T, layer1_delta)

# Plot error against epochs
pd.DataFrame(errors).plot(x="epochs", y="error")
plt.xlabel("Epochs")
plt.ylabel("Mean absolute error")
plt.title("Neural Network Training Error")
plt.show()
```

```
# Final predictions
a1 = sigmoid(np.dot(X, w0))
a2 = sigmoid(np.dot(a1, w1))
print("Final predictions:")
print(a2)
```

3. Explanation of the completed blanks

`a0 = X`
The input layer receives the original feature matrix.

`a1 = sigmoid(np.dot(a0, w0))`
Multiplies the inputs by the first-layer weights, then applies sigmoid to produce the 4 hidden-neuron activations.

`a2 = sigmoid(np.dot(a1, w1))`
Multiplies the hidden activations by the second-layer weights and applies sigmoid to produce the prediction.

`output_error = a2 - y`
Calculates prediction minus actual output. This sign convention is paired with subtracting the weight gradients below.

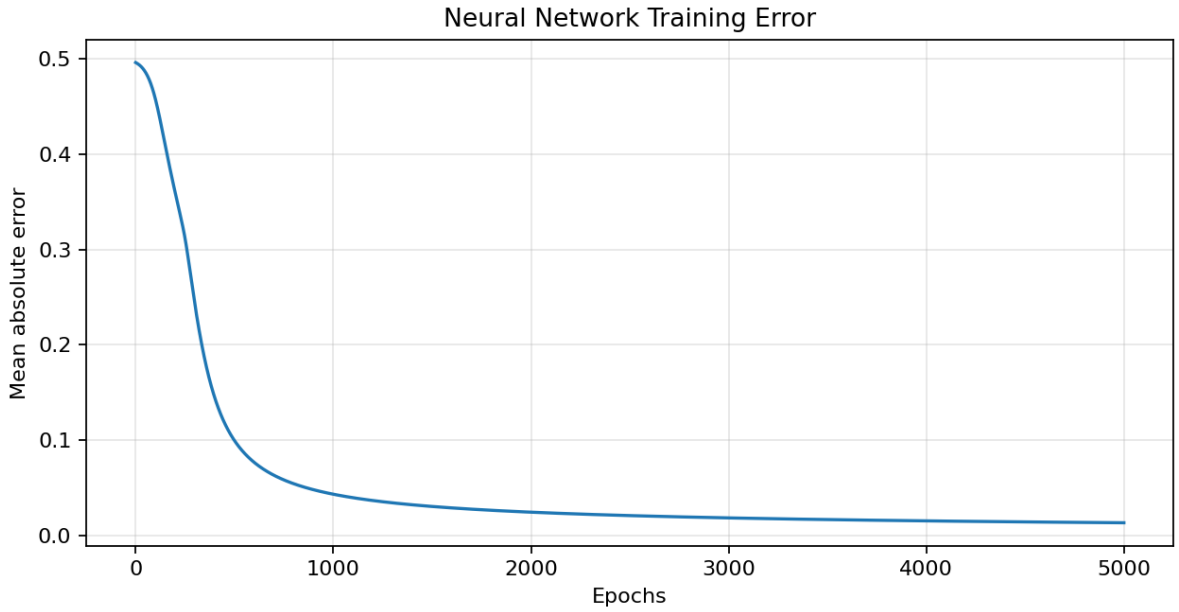
`w1 -= np.dot(a1.T, layer2_delta)`
Updates the hidden-to-output weights using the output-layer delta.

`w0 -= np.dot(a0.T, layer1_delta)`
Updates the input-to-hidden weights using the hidden-layer delta.

4. Training results

The mean absolute error decreases during training, showing that the network is learning the supplied sample patterns.

Epoch	Mean absolute error
0	0.496410
500	0.099292
1000	0.042985
1500	0.030123
2000	0.024122
2500	0.020543
3000	0.018121
3500	0.016353
4000	0.014994
4500	0.013909



5. Final predictions

Input (X)	Predicted output	Expected output
[0, 0, 1]	0.010670	0
[0, 1, 1]	0.986304	1
[1, 0, 1]	0.988221	1
[1, 1, 1]	0.015929	0

The predicted values are close to the target labels 0, 1, 1, 0. Since sigmoid outputs probabilities, values near 0 represent class 0 and values near 1 represent class 1.

6. Key concepts

• **Forward propagation:** computes hidden activations and the final prediction.

• **Error:** measures the difference between predicted and actual output.

• **Backpropagation:** sends output error backward through the network using the sigmoid derivative.

• **Weight update:** adjusts weights to reduce the error.

• **Epoch:** one complete forward and backward pass over the training dataset.